

Lecture 21

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

September 21, 2026

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.
- Example
  $S \rightarrow aAC \quad C_i = A_s$
  $S \rightarrow bABC \quad C_i = A_s$

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.
- Example
  $S \rightarrow aAC \quad C_i = A_s$
  $S \rightarrow bABC \quad C_i = A_s$
  $C \rightarrow c \quad C_s = g(C_i)$

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.
- Example
  $S \rightarrow aAC \quad C_i = A_s$
  $S \rightarrow bABC \quad C_i = A_s$
  $C \rightarrow c \quad C_s = g(C_i)$
- C inherits A_s

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.
- Example
  $S \to aAC \quad C_i = A_s$
  $S \to bABC \quad C_i = A_s$
  $C \to c \quad C_s = g(C_i)$
- C inherits A_s
- There may or may not be a B between A and C on the stack when reduction by rule $C \to c$ takes place

Simulating the evaluation of inherited attributes

- The scheme works only if grammar allows position of attribute to be predicted.
- Example
  $S \rightarrow aAC \quad C_i = A_s$
  $S \rightarrow bABC \quad C_i = A_s$
  $C \rightarrow c \quad C_s = g(C_i)$
- C inherits A_s
- There may or may not be a B between A and C on the stack when reduction by rule $C \rightarrow c$ takes place
- When reduction by $C \rightarrow c$ is performed the value of C_i is either in [top-1] or [top-2]

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to $S \rightarrow aAC \quad C_i = A_s$

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to
  $S \to aAC \quad C_i = A_s$
  $S \to bABMC \quad M_i = A_s; C_i = M_s$

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to
  $S \to aAC \quad C_i = A_s$
  $S \to bABMC \quad M_i = A_s; C_i = M_s$
  $C \to c \quad C_s = g(C_i)$

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to
  $S \to aAC \quad C_i = A_s$
  $S \to bABMC \quad M_i = A_s; C_i = M_s$
  $C \to c \quad C_s = g(C_i)$
  $M \to \epsilon \quad M_s = M_i$

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to
  $S \to aAC \quad C_i = A_s$
  $S \to bABMC \quad M_i = A_s; C_i = M_s$
  $C \to c \quad C_s = g(C_i)$
  $M \to \epsilon \quad M_s = M_i$
- When production $M \to \epsilon$ is applied we have $M_s = M_i = A_s$

Simulating the evaluation

- Insert a marker M just before C in the second rule and change rules to
  $S \rightarrow aAC \quad C_i = A_s$
  $S \rightarrow bABMC \quad M_i = A_s; C_i = M_s$
  $C \rightarrow c \quad C_s = g(C_i)$
  $M \rightarrow \epsilon \quad M_s = M_i$
- When production $M \rightarrow \epsilon$ is applied we have $M_s = M_i = A_s$
- Therefore value of C_i is always at [top-1]

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser
- For every production $A \rightarrow X_1 \cdots X_n$ introduce n markers $M_1 \cdots M_n$ and replace the production by

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser
- For every production $A \to X_1 \cdots X_n$ introduce n markers $M_1 \cdots M_n$ and replace the production by $A \to M_1 X_1 \cdots M_n X_n$

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser
- For every production $A \rightarrow X_1 \cdots X_n$ introduce n markers $M_1 \cdots M_n$ and replace the production by $A \rightarrow M_1 X_1 \cdots M_n X_n$
  $M_1 \rightarrow M_n \rightarrow \epsilon$

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser
- For every production $A \rightarrow X_1 \cdots X_n$ introduce n markers $M_1 \cdots M_n$ and replace the production by $A \rightarrow M_1 X_1 \cdots M_n X_n$
  $M_1 \rightarrow M_n \rightarrow \epsilon$
- Synthesized attribute $X_{j,s}$ goes into the value entry of X_j

General algorithm

- Algorithm: Bottom up parsing and translation with inherited attributes
- Input: L attributed definitions
- Output: A bottom up parser
- For every production $A \rightarrow X_1 \cdots X_n$ introduce n markers $M_1 \cdots M_n$ and replace the production by $A \rightarrow M_1 X_1 \cdots M_n X_n$
  $M_1 \rightarrow M_n \rightarrow \epsilon$
- Synthesized attribute $X_{j,s}$ goes into the value entry of X_j
- Inherited attribute $X_{j,i}$ goes into the value entry of M_j

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production $A \rightarrow M_1 X_1 \cdots M_n X_n$ *then*

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production $A \rightarrow M_1 X_1 \cdots M_n X_n$ *then*
  A_i is in position $top - 2j + 2$

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production $A \rightarrow M_1 X_1 \cdots M_n X_n$ *then*
  A_i is in position $top - 2j + 2$
  $X_{1.i}$ is in position $top - 2j + 3$

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production $A \rightarrow M_1 X_1 \cdots M_n X_n$ *then*
  A_i is in position $top - 2j + 2$
  $X_{1.i}$ is in position $top - 2j + 3$
  $X_{1.s}$ is in position $top - 2j + 4$

Algorithm

- If the reduction is to a marker M_j and the marker belongs to a production $A \rightarrow M_1 X_1 \cdots M_n X_n$ *then*
  A_i is in position $top - 2j + 2$
  $X_{1.i}$ is in position $top - 2j + 3$
  $X_{1.s}$ is in position $top - 2j + 4$
- If reduction is to a non terminal A by production, $A \rightarrow M_1 X_1 \cdots M_n X_n$ then compute A_s and push on the stack

Type System

- A type is a set of values

Type System

- A type is a set of values
- Certain operations are legal for values of each type

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type.

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python
- Static Typing

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python
- Static Typing
  - Catches most common programming errors at compile time

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python
- Static Typing
  - Catches most common programming errors at compile time
  - Avoids runtime overhead

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python
- Static Typing
  - Catches most common programming errors at compile time
  - Avoids runtime overhead
  - May be restrictive in some situations

Type System

- A type is a set of values
- Certain operations are legal for values of each type
- A language's type system specifies which operations are valid for a type
- The aim of type checking is to ensure that operations are used on the variable/expressions of the correct types
- Languages can be divided into three categories with respect to the type:
  - Untyped: No type. Example Assembly
  - Statically typed: type assigned at compile time.Ex C,C++
  - Dynamically typed: type checking at runtime.Ex Python
- Static Typing
  - Catches most common programming errors at compile time
  - Avoids runtime overhead
  - May be restrictive in some situations
  - Rapid prototyping may be difficult

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program
- Different type systems may be used by different compilers for the same language

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program
- Different type systems may be used by different compilers for the same language
- Types of `type`

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program
- Different type systems may be used by different compilers for the same language
- Types of `type`
  - Basic Type: integer, char, float, etc

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program
- Different type systems may be used by different compilers for the same language
- Types of `type`
  - Basic Type: integer, char, float, etc
  - Enumerated type: (voilet,indigo, red)

Type System and Type Checking

- A type system is a collection of rules for assigning type expressions to various parts of a program
- Different type systems may be used by different compilers for the same language
- Types of `type`
  - Basic Type: integer, char, float, etc
  - Enumerated type: (voilet,indigo, red)
  - Constructed Type: array, record, pointers, functions

Type Expression

- Type of a language construct is denoted by a type expression

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or
  it is formed by applying operators called type constructor to other type expressions.

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or
  it is formed by applying operators called type constructor to other type expressions.
- A type constructor applied to a type expression is a type expression

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or
  it is formed by applying operators called type constructor to other type expressions.
- A type constructor applied to a type expression is a type expression
- A basic type is type expression. There are two other special basic types:

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or
  it is formed by applying operators called type constructor to other type expressions.
- A type constructor applied to a type expression is a type expression
- A basic type is type expression. There are two other special basic types:
  - type error: error during type checking

Type Expression

- Type of a language construct is denoted by a type expression
- It is either a basic type
  or
  it is formed by applying operators called type constructor to other type expressions.
- A type constructor applied to a type expression is a type expression
- A basic type is type expression. There are two other special basic types:
  - type error: error during type checking
  - void: no type value

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  var $A : array[1 \cdots 10]$ of integer

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  $var \quad A : array[1 \cdots 10]$ of integer
  A has type expression $array(1 \cdots 10, integer)$

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  $var \quad A : array[1 \cdots 10]$ of integer
  A has type expression $array(1 \cdots 10, integer)$
- Product: if $T1$ and $T2$ are type expressions, then their Cartesian product $T1xT2$ is a type expression

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  $var \quad A : array[1 \cdots 10]$ of integer
  A has type expression $array(1 \cdots 10, integer)$
- Product: if $T1$ and $T2$ are type expressions, then their Cartesian product $T1xT2$ is a type expression
- Pointer: if T is a type expression, then pointer(T) is a type expression denoting type pointer to an object of type T

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  $var \quad A : array[1 \cdots 10]$ of integer
  A has type expression $array(1 \cdots 10, integer)$
- Product: if $T1$ and $T2$ are type expressions, then their Cartesian product $T1xT2$ is a type expression
- Pointer: if T is a type expression, then pointer(T) is a type expression denoting type pointer to an object of type T
- Function: function maps domain set to range set. It is denoted by type expression $D \rightarrow R$

Type Constructor

- Array: if T is a type expression, then $array(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I
  $var \quad A : array[1 \cdots 10]$ of integer
  A has type expression $array(1 \cdots 10, integer)$
- Product: if $T1$ and $T2$ are type expressions, then their Cartesian product $T1xT2$ is a type expression
- Pointer: if T is a type expression, then pointer(T) is a type expression denoting type pointer to an object of type T
- Function: function maps domain set to range set. It is denoted by type expression $D \rightarrow R$
  $function \quad f(char\ a, char\ b) : *integer$; is denoted by $char \times char \rightarrow pointer(integer)$

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration
  `type row = record`

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
```

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
```

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
end;
```

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
end;
var table:  array [1 ..  10] of row;
```

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
end;
var table:  array [1 ..  10] of row;
```

- type expression of table is array($1 \cdots 10$, row)

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
end;
var table:  array [1 ..  10] of row;
```

- type expression of table is array($1 \cdots 10$, row)
- The type row has type expression

Type Constructor

- Records: it applies to a tuple formed from field names and field types. Consider the declaration

```
type row = record
    addr :  integer;
    lexeme :  array [1 ..  15] of char
end;
var table:  array [1 ..  10] of row;
```

- type expression of table is array($1 \cdots 10$, row)
- The type row has type expression
- record ((addr $\times$ integer) $\times$ (lexeme $\times$ *array*($1 \cdots 15$, char)))